

Trabalho 2 – Árvores Binárias e Análise de Complexidade

Grupo: Kaio Henrique da Silva Nezio, Thiago Fabiano e Daniel Pinheiro

Seção 1 - Relato do desenvolvimento das bibliotecas e do programa

Esta seção descreve como o trabalho foi dividido entre os componentes do grupo, onde está o código e como as ferramentas de IA generativa (LLMs) foram usadas.

1.1 Atuação dos componentes do grupo

- **Daniel Pinheiro:** implementou o núcleo da estrutura da árvore binária, criando o arquivo No.java e o esqueleto da classe ArvoreBinaria, incluindo o construtor com a interface Comparator. Desenvolveu os métodos centrais adicionar (inserindo a restrição para não aceitar chaves duplicadas), pesquisar e quantidadeNos. Na parte de documentação, realizou a atualização do README e escreveu a Seção 1 do relatório. Também foi responsável pela introdução da Seção 2 (explicando o comportamento da altura da árvore nos cenários balanceado e degenerado) e pelas análises das Seções 2.1 (pesquisar), 2.2 (adicionar) e 2.4 (quantidadeNos).
- **Thiago Fabiano:** focou na lógica de remoção e nos percursos da estrutura. Implementou o método remover, tratando adequadamente os três casos possíveis (nó folha, com um filho e com dois filhos, utilizando o sucessor). Desenvolveu os métodos altura, caminharEmOrdem e caminharEmNivel (utilizando estrutura de fila para impressão de uma linha por nível). Atualizou o Makefile e o README e refez os testes da seção 3. No relatório, fez análise do método de remoção na Seção 2.3 e elaborou a parte final da Seção 3.2, comparando os tempos de execução da árvore com os das listas encadeadas do trabalho anterior.
- **Kaio Henrique:** ficou encarregado da aplicação principal, dos geradores de dados e da condução dos experimentos. Estruturou a classe Main com as três opções de execução exigidas, removendo os casts diretos para ListaEncadeada. Desenvolveu os programas GeradorArquivosOrdenados e GeradorArquivosBalanceados para o contexto de contatos e implementou a lógica de busca para encontrar o elemento de pior caso. Realizou a primeira coleta de tempos com os oito arquivos. No relatório, escreveu integralmente a Seção 3.1 (detalhando os geradores e a escolha do pior caso), montou

as tabelas de resultados na Seção 3.2 e elaborou a interpretação empírica dos tempos obtidos em contraponto à teoria apresentada na Seção 2.

1.2 Repositório e organização do código

O código e este relatório estão no repositório do GitHub <https://github.com/ThiagoFabiano/bsi-tpa-trabalhos/tree/trabalho2>. É o mesmo repositório do trabalho 1 porém o trabalho 2 está todo na branch “trabalho2”. O README do repositório descreve a organização do código e como executá-lo.

1.3 Uso de LLMs no desenvolvimento

O grupo utilizou ferramentas de modelos de linguagem (LLMs), como Gemini e Claude, como apoio durante o desenvolvimento do projeto e a elaboração do relatório. Seu uso esteve voltado principalmente à consulta, revisão, esclarecimento de dúvidas técnicas e apoio à organização do trabalho.

De forma mais detalhada, as ferramentas foram utilizadas para:

- auxiliar na análise e revisão de trechos de código relacionados às etapas 1 e 2, incluindo a ArvoreBinaria, adaptações na Main, ContatoController, ContatoService e os geradores de arquivos;
- auxiliar na identificação e compreensão de erros de compilação relacionados ao uso de tipos genéricos sem parâmetro (raw types), como IColecao em vez de IColecao<Contato>, Medicao sem <T> e List sem <Long>, presentes no ContatoService, na Main e no GeradorArquivosBalanceados;
- apoiar a revisão da lógica da ArvoreBinaria, considerando aspectos como inserção, busca, os três casos de remoção (folha, um filho e dois filhos), cálculo da altura e caminhamentos iterativos. A análise realizada não identificou problemas de lógica nesses pontos;
- apoiar a definição do guia de testes e do protocolo de coleta de tempos utilizados na seção 3;
- auxiliar na elaboração e revisão do relatório, especialmente na organização das análises de complexidade, estruturação das tabelas e revisão textual.
- Criação do arquivo GeradorNomes.java e Makefile

2 Análise matemática de complexidade dos algoritmos

Esta seção analisa os métodos *pesquisar*, *adicionar*, *remover* e *quantidadeNos* da classe *ArvoreBinaria*, nessa ordem: a descida pela árvore é explicada uma vez no *pesquisar* e reaproveitada no *adicionar* e no *remover*. As listagens têm as linhas numeradas, e a análise cita esses números. Em todas as contas, n é o número de contatos na árvore e h é a altura da

árvore, isto é, o número de ligações do caminho mais longo entre a raiz e uma folha; uma árvore só com a raiz tem $h = 0$, e o caminho mais longo passa por $h + 1$ nós. Cada método é analisado no pior caso, em duas árvores extremas: a totalmente degenerada ($h = n - 1$) e a perfeitamente balanceada, em que todos os níveis estão cheios: o nível 0 tem 1 nó, o nível 1 tem 2, e cada nível tem o dobro do anterior, até o nível h , que tem 2^h nós. Somando os níveis:

$$n = 1 + 2 + 4 + \dots + 2^h = 2^{(h+1)} - 1, \text{ logo } h = \log_2(n + 1) - 1 \approx \log_2 n$$

A altura da árvore balanceada cresce com o logaritmo de n : dobrar a quantidade de contatos acrescenta só um nível, ou seja, uma comparação a mais. Com os tamanhos de arquivo usados na seção 3, o caminho mais longo tem 16 nós com 50.000 contatos, 17 com 100.000, 18 com 200.000 e 19 com 400.000.

2.1 Método pesquisar

O pesquisar procura um valor na árvore e devolve o objeto guardado nela, ou null se não achar. Devolver o objeto guardado, e não a chave recebida, é o que permite ao sistema achar um contato inteiro a partir de uma chave parcial: o *ContatoService* pesquisa com um Contato que só tem o telefone preenchido e recebe de volta o contato completo, com o nome. A Listagem 1 mostra o método.

```
1@Override
2public T pesquisar(T valor) {
3    if (valor == null) {
4        return null;
5    }
6    No<T> atual = this.raiz;
7    while (atual != null) {
8        int comp = this.comparador.compare(valor, atual.getValor());
9        if (comp == 0) {
10            return atual.getValor();
11        } else if (comp < 0) {
12            atual = atual.getEsq();
13        } else {
14            atual = atual.getDir();
15        }
16    }
17    return null;
18}
```

Listagem 1 – método pesquisar

2.1.1 A busca (linhas 3 a 17)

As linhas 3 a 5 recusam o valor nulo e a linha 6 começa pela raiz. O while das linhas 7 a 16 desce um nível por volta: compara o valor procurado com o do nó atual (linha 8); se forem iguais, devolve o valor guardado (linhas 9 e 10); se o procurado for menor, desce para a

esquerda (linhas 11 e 12); se for maior, desce para a direita (linhas 13 e 14). Se o atual ficar nulo, a árvore acabou sem que o valor aparecesse, e a linha 17 devolve null.

Aqui está a grande diferença para a lista do Trabalho 1. Na lista, a busca passava pelos nós um atrás do outro. Na árvore, a cada volta o método escolhe um lado e deixa a outra subárvore inteira para trás, sem olhar para ela. Por isso o número de voltas não depende do total de nós, e sim de quantos nós existem no caminho da raiz até o valor procurado: no máximo $h + 1$. O adicionar e o remover fazem essa mesma descida.

2.1.2 Somando tudo

Linha	O que faz	Quantas vezes roda no pior caso
3 a 6	recusa valor nulo e começa pela raiz	1
7	testa se ainda há nó para olhar	$h + 2$
8 e 11 a 14	compara com o nó atual e desce um nível	$h + 1$
9 e 10	devolve o valor se o achou	1
17	devolve null se o valor não existe	1

Tabela – contagem de execuções do método pesquisar

$$T(n) = c_1 \cdot h + c_2, \text{ então o método é } O(h)$$

2.1.3 Pior caso nas duas árvores

Na árvore balanceada, $h \approx \log_2 n$ (ver a introdução da seção), então $T(n) = c_1 \cdot \log_2(n) + c_2$ e o método é $O(\log n)$. O pior caso é pesquisar uma folha do último nível, ou um valor que não existe: a busca desce a árvore inteira, com no máximo 16 a 19 comparações nos tamanhos de arquivo usados. Na árvore degenerada, $h = n - 1$, então $T(n) = c_1 \cdot (n - 1) + c_2$ e o método é $O(n)$. O pior caso é pesquisar o último contato inserido, o de maior telefone, ou um telefone maior que todos: a busca passa pelos n nós. É esse contato que usamos nos testes da seção 3. O melhor caso é o primeiro contato inserido, que é a raiz e custa uma comparação.

2.1.4 Resumo

Formato da árvore	Altura h	pesquisar no pior caso
Perfeitamente balanceada	$\approx \log_2 n$	$O(\log n)$
Degenerada	$n - 1$	$O(n)$
Lista encadeada (Trabalho 1)	—	$O(n)$

Tabela – ordem de complexidade do pesquisar conforme o formato da árvore

2.2 Método adicionar

O adicionar insere um contato novo no lugar certo da árvore. Ele recusa o valor nulo e também o valor repetido, isto é, aquele que o comparador considera igual a um nó que já existe; nos dois casos devolve false e a árvore não muda. Caso contrário, desce da raiz até achar uma

posição vazia e pendura o nó novo ali, como folha. Não há rebalanceamento nenhum: o formato da árvore depende só da ordem em que os valores chegam. A Listagem 2 mostra o método.

```
1 @Override
2 public boolean adicionar(T novoValor) {
3     if (novoValor == null) {
4         return false;
5     }
6     No<T> novo = new No<>(novoValor);
7     if (this.raiz == null) {
8         this.raiz = novo;
9         this.quantidade++;
10        return true;
11    }
12    No<T> atual = this.raiz;
13    while (true) {
14        int comp = this.comparador.compare(novoValor, atual.getValor());
15        if (comp == 0) {
16            return false;
17        }
18        if (comp < 0) {
19            if (atual.getEsq() == null) {
20                atual.setEsq(novo);
21                break;
22            }
23            atual = atual.getEsq();
24        } else {
25            if (atual.getDir() == null) {
26                atual.setDir(novo);
27                break;
28            }
29            atual = atual.getDir();
30        }
31    }
32    this.quantidade++;
33    return true;
34}
```

Listagem 2 – método adicionar

2.2.1 Os casos simples (linhas 3 a 12)

As linhas 3 a 5 recusam o valor nulo. A linha 6 cria o nó novo, que ainda não está ligado à árvore. Se a árvore está vazia (linha 7), o nó novo vira a raiz, o contador aumenta e o método devolve true (linhas 8 a 10). A linha 12 prepara a descida: o atual começa pela raiz. Essas linhas rodam no máximo uma vez e levam tempo fixo.

2.2.2 A descida (linhas 13 a 33)

O while (true) da linha 13 não tem condição de parada no cabeçalho; ele termina por um return ou por um break no corpo. Ele faz a mesma descida do pesquisar (seção 2.1.1): compara o valor novo com o do nó atual (linha 14) e, se forem iguais, devolve false (linhas 15 a 17); se o novo for menor, vai para a esquerda (linhas 18 a 23), e se for maior, para a direita (linhas 24 a 29). A diferença é o que acontece quando o filho daquele lado não existe: o nó novo é pendurado ali (linha 20 ou 26) e o laço termina com break. Depois do laço, as linhas 32 e 33 aumentam o contador e devolvem true. Como no pesquisar, o laço dá no máximo $h + 1$ voltas, sendo h a altura que a árvore tem antes da inserção.

2.2.3 Somando tudo

Linha	O que faz	Quantas vezes roda no pior caso
3 a 6	recusa valor nulo e cria o nó novo	1
7 a 11	trata a árvore vazia	1
12	começa a descida pela raiz	1
13 e 14	repete e compara com o nó atual	$h + 1$
15 a 17	devolve false se o valor já existe	1
18 a 30	escolhe o lado, desce ou pendura o nó novo	$h + 1$
32 e 33	atualiza o contador e devolve true	1

Tabela – contagem de execuções do método adicionar

O trabalho é uma quantidade fixa de comandos por volta, mais um resto que é tempo fixo:

$$T(n) = c_1 \cdot h + c_2, \text{ então o método é } O(h)$$

2.2.4 Pior caso nas duas árvores

Na árvore balanceada, $h \approx \log_2 n$ e o método é $O(\log n)$. Todos os lugares vazios ficam abaixo das folhas do último nível, então qualquer valor novo desce a árvore inteira antes de ser pendurado: o pior caso é qualquer inserção de valor novo, com $h + 1$ comparações. Montar a árvore com n inserções custa, então, $O(n \log n)$. É o que acontece com os arquivos balanceados.

Na árvore degenerada, $h = n - 1$ e o método é $O(n)$. Quando os telefones entram em ordem crescente, cada contato novo é maior que todos os que já estão na árvore, e a descida vai sempre para a direita até o fim. Montar a árvore inteira soma todas as descidas: $0 + 1 + 2 + \dots + (n - 1) = n \cdot (n - 1) / 2$ comparações, ou seja, $O(n^2)$. Com os tamanhos de arquivo que usamos, isso dá cerca de 1,25 bilhão de comparações com 50.000 contatos e cerca de 80 bilhões com 400.000: dobrar o n multiplica o trabalho por quatro. O melhor caso do adicionar é a inserção na árvore vazia, em tempo fixo.

2.2.5 Resumo

Estrutura	adicionar no pior caso	Montagem com n contatos
Árvore perfeitamente balanceada	$O(\log n)$	$O(n \log n)$
Árvore degenerada	$O(n)$	$O(n^2)$
Lista não ordenada (Trabalho 1)	$O(1)$	$O(n)$
Lista ordenada (Trabalho 1)	$O(n)$	$O(n^2)$

Tabela – ordem de complexidade do adicionar conforme a estrutura

2.3 Método remover

O remover é o método mais longo da árvore. Além de achar o nó, ele precisa tirá-lo sem quebrar a regra da árvore de busca: à esquerda de um nó só ficam valores menores, e à direita só valores maiores. O método tem duas partes. Primeiro ele busca o nó, como no pesquisar, mas guardando também o pai. Depois ele retira o nó, e o jeito de retirar depende de quantos filhos o nó tem. A Listagem 3 mostra o remover e a Listagem 4 mostra o trocarFilho, um método auxiliar que liga o pai ao nó que vai ficar no lugar do removido.

```

1 @Override
2 public boolean remover(T valor) {
3     if (valor == null) {
4         return false;
5     }
6     No<T> pai = null;
7     No<T> atual = this.raiz;
8     while (atual != null) {
9         int comp = this.comparador.compare(valor, atual.getValor());
10        if (comp == 0) {
11            break;
12        }
13        pai = atual;
14        atual = (comp < 0) ? atual.getEsq() : atual.getDir();
15    }
16    if (atual == null) {
17        return false;
18    }
19
20    if (atual.getEsq() == null && atual.getDir() == null) {
21        trocarFilho(pai, atual, null);
22    } else if (atual.getEsq() == null || atual.getDir() == null) {
23        No<T> filho = (atual.getEsq() != null) ? atual.getEsq() : atual.getDir();
24        trocarFilho(pai, atual, filho);
25    } else {
26        No<T> paiSucessor = atual;
27        No<T> sucessor = atual.getDir();
28        while (sucessor.getEsq() != null) {
29            paiSucessor = sucessor;
30            sucessor = sucessor.getEsq();
31        }
32        atual.setValor(sucessor.getValor());
33        trocarFilho(paiSucessor, sucessor, sucessor.getDir());
34    }
35    this.quantidade--;
36    return true;
37 }

```

Listagem 3 – método remover

```

1 private void trocarFilho(No<T> pai, No<T> antigo, No<T> novo) {
2     if (pai == null) {
3         this.raiz = novo;
4     } else if (pai.getEsq() == antigo) {
5         pai.setEsq(novo);
6     } else {
7         pai.setDir(novo);
8     }
9 }

```

Listagem 4 – método auxiliar trocarFilho

2.3.1 A busca do nó (linhas 3 a 18)

A busca é igual à do pesquisar (seção 2.1.1), com uma diferença: o método guarda o pai do nó atual, porque vai precisar dele para religar a árvore. As linhas 6 e 7 começam pela raiz, com o pai nulo. No while das linhas 8 a 15, a linha 9 compara o valor procurado com o do nó atual. Se forem iguais, o laço para (linhas 10 a 12). Se não forem, o atual vira o pai e o método desce para a esquerda ou para a direita (linhas 13 e 14). Se o atual ficar nulo, o valor não está na árvore e as linhas 16 a 18 devolvem false. O laço dá no máximo $h + 1$ voltas.

2.3.2 A retirada do nó (linhas 20 a 34)

Quando acha o nó, o método vê quantos filhos ele tem. São três casos:

- **Caso 1 – folha (linhas 20 e 21):** o nó não tem filhos. Basta o pai deixar de apontar para ele, e o trocarFilho coloca null no lugar do nó.
- **Caso 2 – um filho (linhas 22 a 24):** a linha 23 vê de que lado está o filho, e o trocarFilho coloca esse filho no lugar do nó removido. A subárvore do filho sobe junto e continua em ordem, porque todos os valores dela já estavam do mesmo lado do pai.
- **Caso 3 – dois filhos (linhas 25 a 34):** não dá para subir um dos filhos, porque o outro ficaria sem lugar. Então o método troca o valor do nó pelo valor do sucessor, que é o menor valor da subárvore direita (o que vem logo depois no caminhar em ordem). Para achar o sucessor, o método vai uma vez para a direita (linha 27) e depois sempre para a esquerda até acabar (linhas 28 a 31). A linha 32 copia o valor do sucessor para o nó, e a linha 33 tira o sucessor do lugar dele. Como o sucessor não tem filho esquerdo, essa retirada é sempre um caso 1 ou um caso 2.

Isso funciona porque o sucessor é maior que todos os valores da subárvore esquerda e menor que os outros valores da subárvore direita, então a regra da árvore continua valendo. Daria para usar o antecessor (o maior valor da subárvore esquerda) do mesmo jeito; nós escolhemos o sucessor.

O trocarFilho (Listagem 4) não tem laço, só um if-else. Se não há pai (linha 2), o nó retirado era a raiz, e a raiz passa a ser o novo nó (linha 3). Se há pai, ele troca o filho esquerdo ou o direito, dependendo do lado em que o nó antigo estava (linhas 4 a 7). Ele sempre leva o mesmo tempo, $O(1)$. No fim do remover, as linhas 35 e 36 diminuem o contador e devolvem true, também em tempo fixo.

2.3.3 Somando tudo

Linha	O que faz	Quantas vezes roda no pior caso
3 a 7	recusa valor nulo e começa a busca pela raiz	1
8	testa se ainda há nó para olhar	$h + 2$
9 a 14	compara com o nó atual e desce um nível	$h + 1$
16 a 18	devolve false se o valor não existe	1

Linha	O que faz	Quantas vezes roda no pior caso
20 a 24	casos 1 e 2: religa o pai	1
26, 27, 32 e 33	caso 3: começa a busca do sucessor, copia o valor e religa	1
28 a 31	caso 3: desce até o sucessor	junto com as linhas 8 a 15, no máximo $h + 1$
35 e 36	atualiza o contador e devolve true	1

Tabela – contagem de execuções do método remover

O caso 3 tem dois laços, mas eles não somam $2h$ voltas. O primeiro desce da raiz até o nó que vai ser removido, e o segundo continua descendo dali até o sucessor. Os dois juntos fazem um único caminho da raiz até o sucessor, e nenhum caminho da árvore tem mais de $h + 1$ nós. Então, nos três casos, o trabalho depende só da altura, e o resto do método é tempo fixo:

$$T(n) = c_1 \cdot h + c_2, \text{ então o método é } O(h)$$

2.3.4 Pior caso na árvore perfeitamente balanceada

Com $h \approx \log_2 n$ (dedução na introdução da seção 2), substituindo na conta do remover:

$$T(n) = c_1 \cdot \log_2(n) + c_2, \text{ então o método é } O(\log n) \text{ (logarítmico)}$$

O pior caso é remover uma folha do último nível, porque a busca desce a árvore inteira até ela. Procurar um valor que não existe dá o mesmo trabalho. Remover a raiz também chega perto disso: a busca para na primeira volta, mas o sucessor da raiz está lá embaixo, e o laço das linhas 28 a 31 desce até ele.

2.3.5 Pior caso na árvore degenerada

Quando os contatos entram em ordem crescente de telefone, cada contato novo é maior que todos os anteriores e vai sempre para a direita do último. Cada nó fica com um só filho, o direito, e a árvore vira na prática uma lista encadeada. A altura é a maior possível:

$$h = n - 1, \text{ então } T(n) = c_1 \cdot (n - 1) + c_2 \text{ e o método é } O(n) \text{ (linear)}$$

O pior caso é remover o último contato inserido, o de maior telefone, que é a folha no fim da árvore: a busca compara com todos os n contatos. Procurar um telefone que não existe e é maior que todos também passa pelos n nós. Nessa árvore o caso 3 nunca acontece, porque nenhum nó tem dois filhos. O melhor caso é remover a raiz: o laço dá uma volta só e o filho direito sobe para o lugar dela.

2.3.6 Por que o remover usa laço e não recursão

Os métodos da árvore usam laços, e não recursão. Na árvore degenerada de 400.000 contatos, a recursão abriria 400.000 chamadas ao mesmo tempo e o programa pararia com um `StackOverflowError`. A ordem de complexidade é a mesma nas duas versões.

2.3.7 Resumo

Formato da árvore	Altura h	remover no pior caso
Perfeitamente balanceada	$\approx \log_2 n$	$O(\log n)$
Degenerada	$n - 1$	$O(n)$
Lista encadeada (Trabalho 1)	—	$O(n)$

Tabela – ordem de complexidade do remover conforme o formato da árvore

O código é o mesmo nos dois casos; o que muda é o formato da árvore. Balanceada, a remoção é muito mais rápida que na lista. Degenerada, ela fica igual à lista.

2.4 Método quantidadeNos

O quantidadeNos informa quantos contatos a árvore guarda. A Listagem 5 mostra o método.

```
1 @Override
2 public int quantidadeNos() {
3     return this.quantidade;
4 }
```

Listagem 5 – método quantidadeNos

O método não percorre a árvore: ele devolve o atributo quantidade (linha 3), que a própria árvore mantém atualizado. O adicionar soma 1 ao contador quando insere um nó novo (linhas 9 e 32 da Listagem 2), e o remover subtrai 1 quando retira um nó (linha 35 da Listagem 3). Quando o adicionar recusa um valor nulo ou repetido, ele devolve false antes de chegar a essas linhas, e quando o remover não acha o valor, ele também devolve false sem mexer no contador. Assim o contador só muda quando a quantidade de nós de fato muda.

O método tem um único comando, que lê um atributo e devolve. Não há laço e nada depende do formato da árvore:

$$T(n) = c, \text{ então o método é } O(1) \text{ (constante)}$$

Isso vale tanto na árvore perfeitamente balanceada quanto na degenerada.

2.5 Resumo dos quatro métodos

Método	Perfeitamente balanceada	Degenerada	Quando ocorre o pior caso
adicionar	$O(\log n)$	$O(n)$	valor novo que desce até o último nível; na degenerada, valor maior que todos
pesquisar	$O(\log n)$	$O(n)$	folha do último nível ou valor inexistente; na degenerada, o último inserido
remover	$O(\log n)$	$O(n)$	folha do último nível ou valor inexistente; na degenerada, o último inserido
quantidadeNos	$O(1)$	$O(1)$	não há pior caso: o tempo é sempre o mesmo

Tabela – ordem de complexidade dos métodos no pior caso

3 Análise empírica de complexidade dos algoritmos da biblioteca

Esta seção mede o tempo de execução do sistema usando árvores binárias, com arquivos de quatro tamanhos (50.000, 100.000, 200.000 e 400.000 contatos), os mesmos do Trabalho 1. Para cada tamanho há dois arquivos: um com os telefones em ordem crescente, que gera uma árvore degenerada, e um com os contatos ordenados de modo a gerar uma árvore perfeitamente balanceada.

3.1 Implementação

3.1.1 Geração dos arquivos de entrada

Cada linha dos arquivos tem o formato nome;telefone. Os dois geradores, *GeradorArquivosOrdenados* e *GeradorArquivosBalanceados* (pasta *util*), foram adaptados dos exemplos do professor para o domínio do sistema: em vez de alunos com matrícula, geram contatos com nome e telefone. O telefone é 27900000000 mais um índice de 0 a $n - 1$, nos dois geradores. O nome (“Nome Sobrenome Sobrenome”) é sorteado pela classe *GeradorNomes* a partir das listas do gerador do professor, e um *HashSet* impede repetições. Isso ocorre só na geração dos arquivos e não afeta as medições.

Arquivo ordenado. Os índices são escritos em ordem crescente. Inseridos nessa ordem, cada telefone é maior que todos os anteriores e vai para a direita do último, o que gera a árvore degenerada, de altura $n - 1$.

Arquivo balanceado. Os índices são escritos pelo método recursivo *dividirEConquistar*. Para o intervalo de índices que resta, ele escreve primeiro o índice do meio, que será a raiz daquele trecho, e depois trata a metade esquerda e a metade direita do mesmo jeito, até o intervalo acabar. Inserida nessa ordem, a árvore fica perfeitamente balanceada, com altura igual à parte inteira de $\log_2 n$. A recursão do gerador tem profundidade de cerca de $\log_2 n$, então não há risco de estourar a pilha.

Tamanho	Altura da árvore balanceada	Nós no caminho mais longo	Altura da árvore degenerada
50.000	15	16	49.999
100.000	16	17	99.999
200.000	17	18	199.999
400.000	18	19	399.999

Tabela – alturas esperadas das árvores de telefone

Conferido: inserindo os arquivos balanceados numa *ArvoreBinaria*, *altura()* deu 15, 16, 17 e 18. Nos oito arquivos, as duas árvores ficaram com o mesmo número de nós do arquivo.

Nomes únicos. A árvore recusa chaves repetidas. Com os 440 nomes da versão antiga do gerador, a árvore de nomes ficava com 440 nós e a pesquisa por nome devolvia outro contato.

Com nomes únicos, ela guarda os n contatos; como os nomes chegam em ordem aleatória, sua altura ficou entre 34 e 47.

3.1.2 Definição do elemento de pior caso

O enunciado pede a busca pela folha mais distante da raiz na árvore indexada por telefone. O mesmo contato é usado nas três medidas (pesquisa por telefone, pesquisa pelo nome dele e remoção) e, como os nomes são únicos, a pesquisa por nome acha exatamente esse contato. Ele foi definido assim:

- **Arquivo ordenado:** o contato de maior telefone, que é o último do arquivo. Ele é a folha no fim da corrente, a $n - 1$ ligações da raiz, e a busca passa pelos n nós.
- **Arquivo balanceado:** o contato de uma folha do último nível, a uma distância da raiz igual à altura da árvore. Todas as folhas do último nível dão o mesmo trabalho à busca. O gerador acompanha a profundidade da recursão e guarda o primeiro telefone que escreve na maior profundidade e o mostra no terminal (“Telefone de Pior Caso: ...”): 27900000001 em todos os tamanhos. O gerador de ordenados também mostra o seu, o último do arquivo.

Arquivo	Telefone de pior caso	Nome do contato
ordenado, 50.000	27900049999	Igor Paz Queiroz
ordenado, 100.000	27900099999	Renato Abreu Vasconcelos
ordenado, 200.000	27900199999	Jorge Xisto Lima
ordenado, 400.000	27900399999	Emanuel Iglesias Lima
balanceado, 50.000	27900000001	Mauricio Porto Werneck
balanceado, 100.000	27900000001	Osvaldo Vilela Delgado
balanceado, 200.000	27900000001	Gabriel Dantas Xisto
balanceado, 400.000	27900000001	Renata Noronha Rangel

Tabela – elemento de pior caso de cada arquivo

3.1.3 O que cada tempo mede

A **montagem** é o tempo do carregarArquivo inteiro: leitura do arquivo, divisão de cada linha e inserção de cada contato nas duas árvores, a de nome e a de telefone. Nas outras três medidas só a chamada ao método da árvore é cronometrada: o pesquisar da árvore de telefones (**pesquisa por telefone**), o pesquisar da árvore de nomes (**pesquisa por nome**), as duas na segunda chamada (ver 3.1.4), e o remover da árvore de telefones (**remoção por telefone**). A pesquisa do contato completo que vem antes da remoção e a retirada da árvore de nomes ficam fora. Os tempos são lidos com System.nanoTime() e exibidos em nanossegundos (ns).

3.1.4 Procedimento de coleta

Item	Valor
Processador	AMD Ryzen 7 5825U (8 núcleos, 16 threads, até 4,5 GHz)
Memória RAM	16 GB
Sistema operacional	Ubuntu 24.04.4 LTS
Versão do Java	OpenJDK 21.0.12.1
Mesma máquina do Trabalho 1	Sim

Tabela – ambiente de teste

Para cada um dos oito arquivos, os passos foram:

1. iniciar o programa numa JVM nova e escolher a opção 3 (árvore binária);
2. na opção 1 do menu, informar o caminho do arquivo e anotar o tempo de montagem e o tamanho das duas estruturas;
3. na opção 4, pesquisar o telefone de pior caso duas vezes seguidas, anotar os dois tempos e o nome exibido;
4. na opção 3, pesquisar esse nome duas vezes seguidas e anotar os dois tempos;
5. na opção 5, remover o telefone de pior caso e anotar o tempo;
6. na opção 7, encerrar o programa.

Cada arquivo foi executado 3 vezes, sempre numa JVM nova, e as tabelas trazem a mediana. Um script digitou as opções do menu no programa, sem alterá-lo. Cada rodada levou cerca de uma hora, quase toda na montagem degenerada de 400.000.

As pesquisas são feitas duas vezes porque a primeira paga o aquecimento da JVM (balanceada de 400.000: 213.352 ns na primeira, 3.086 ns na segunda); as tabelas usam a segunda. A remoção não pode ser repetida, por isso inclui esse custo.

3.2 Análise de complexidade

3.2.1 Tempos medidos

Tamanho	Altura h	Montagem (ms)	Pesquisa por telefone (ns)	Pesquisa por nome (ns)	Remoção por telefone (ns)
50.000	15	95	2.776	25.728	7.875
100.000	16	161	3.296	27.362	7.825
200.000	17	307	3.086	26.108	8.326
400.000	18	662	3.086	15.850	8.526

Tabela – tempos medidos na árvore balanceada (mediana de 3 execuções)

Tamanho	Altura h	Montagem (s)	Pesquisa por telefone (ns)	Pesquisa por nome (ns)	Remoção por telefone (ns)
50.000	49.999	14,6	1.756.150	38.563	3.244.465

Tamanho	Altura h	Montagem (s)	Pesquisa por telefone (ns)	Pesquisa por nome (ns)	Remoção por telefone (ns)
100.000	99.999	99,2	4.553.259	27.181	9.252.237
200.000	199.999	560,7	9.958.482	28.243	15.589.482
400.000	399.999	3.060,5	18.985.797	29.326	24.554.280

Tabela – tempos medidos na árvore degenerada (mediana de 3 execuções)

3.2.2 O que esperamos da seção 2

Como o tamanho dobra de um arquivo para o seguinte, a razão entre dois tempos consecutivos mostra a ordem de complexidade: uma razão perto de 1 indica tempo quase constante, perto de 2 indica crescimento linear e perto de 4, crescimento quadrático. A seção 2 prevê o seguinte.

Medida	Balanceda	Razão esperada	Degenerada	Razão esperada
Montagem	$O(n \log n)$	$\approx 2,1$	$O(n^2)$	≈ 4
Pesquisa por telefone	$O(\log n)$	$\approx 1,06$	$O(n)$	≈ 2
Remoção por telefone	$O(\log n)$	$\approx 1,06$	$O(n)$	≈ 2
Pesquisa por nome	ver 3.2.4	—	ver 3.2.4	—

Tabela – comportamento esperado ao dobrar o tamanho do arquivo

A razão 1,06 vem de passar de 16 para 17, de 17 para 18 e de 18 para 19 comparações. É uma diferença tão pequena que, na prática, esperamos que os tempos da árvore balanceada oscilem sem crescer de forma clara.

3.2.3 Razões medidas

Medida	Árvore	50 mil → 100 mil	100 mil → 200 mil	200 mil → 400 mil
Montagem	balanceada	1,69	1,91	2,16
Montagem	degenerada	6,81	5,65	5,46
Pesquisa por telefone	balanceada	1,19	0,94	1,00
Pesquisa por telefone	degenerada	2,59	2,19	1,91
Remoção por telefone	balanceada	0,99	1,06	1,02
Remoção por telefone	degenerada	2,85	1,68	1,58
Pesquisa por nome	balanceada	1,06	0,95	0,61
Pesquisa por nome	degenerada	0,70	1,04	1,04

Tabela – razão entre os tempos de tamanhos consecutivos

Árvore balanceada. A montagem teve razões 1,69, 1,91, 2,16, perto do $\approx 2,1$ de $O(n \log n)$. A pesquisa por telefone (2,8 a 3,3 μs) e a remoção (7,8 a 8,5 μs) ficaram praticamente constantes, com razões perto de 1, como previsto para $O(\log n)$: de 50.000 para 400.000 contatos a busca ganha só 3 comparações. Esses tempos são tão pequenos que quase tudo neles é o custo de chamar e cronometrar o método, e não as comparações em si: 19

comparações em uns 3 μ s dão uns 160 ns por comparação, enquanto na árvore degenerada cada nó custou uns 40 ns. Por isso, o que mostra que a busca é $O(\log n)$ é o tempo não crescer com n , e não o valor medido.

Árvore degenerada. A pesquisa por telefone teve razões 2,59, 2,19, 1,91, perto do ≈ 2 de $O(n)$. A remoção cresceu 7,6 vezes para um arquivo 8 vezes maior, também linear; suas razões variam mais porque ela é a primeira execução do remover e inclui o aquecimento da JVM.

Na montagem degenerada as razões (6,81, 5,65, 5,46) ficaram acima do 4 de $O(n^2)$. O número de comparações segue a conta $n \cdot (n - 1)/2$, mas cada uma ficou mais cara: cerca de 12 ns com 50.000 contatos e 38 ns com 400.000. Com 50.000 contatos os dados cabem na memória cache do processador; com 400.000 não cabem, e cada passo da descida busca dados na memória principal, mais lenta. A pesquisa na árvore degenerada mostra o mesmo efeito: o tempo gasto em cada nó subiu de uns 35 ns com 50.000 contatos para uns 47 ns com 400.000. A ordem continua $O(n^2)$; muda só a constante.

3.2.4 A pesquisa por nome

A pesquisa por nome roda na árvore de nomes. Como os nomes são sorteados, eles chegam em ordem aleatória, seja qual for a ordem dos telefones, e a árvore de nomes fica perto de uma árvore balanceada, sem degenerar. Além disso, o contato escolhido é o pior caso para o telefone, mas não para o nome: a posição dele na árvore de nomes é sorteada.

A pesquisa por nome ficou entre 15,8 e 38,6 μ s em todos os arquivos, sem crescer com n e sem depender da ordem dos telefones: com 400.000 contatos na árvore degenerada, a pesquisa por telefone levou 19,0 ms e a por nome, do mesmo contato, 29,3 μ s. Ela é mais lenta que a pesquisa por telefone na balanceada (3,1 μ s), em parte porque a árvore de nomes é mais alta (34 a 47 níveis) e `compareToIgnoreCase` é mais cara que comparar telefones. Mas a diferença, de 5 a 10 vezes, parece grande demais para ser só isso. O tempo também depende de onde o contato caiu na árvore de nomes, o que é sorteado, e não medimos qual dos motivos pesa mais.

3.2.5 Comparação com as listas do Trabalho 1

No Trabalho 1 medimos os mesmos tempos com listas encadeadas, usando arquivos dos mesmos tamanhos (50.000, 100.000, 200.000 e 400.000 contatos). Nos dois trabalhos, a busca e a remoção foram feitas no pior caso de cada estrutura: na lista, o último contato; na árvore, a folha mais longe da raiz. As tabelas abaixo juntam os tempos das listas, copiados do relatório do Trabalho 1, com os das árvores, medidos na mesma máquina. Como no Trabalho 1 cada pesquisa foi medida uma vez só, aqui usamos a primeira pesquisa das árvores.

Tamanho	Lista não ordenada (ms)	Lista ordenada (s)	Árvore balanceada (ms)	Árvore degenerada (s)
50.000	43,4	46,93	95	14,6
100.000	91,1	205,99	161	99,2

Tamanho	Lista não ordenada (ms)	Lista ordenada (s)	Árvore balanceada (ms)	Árvore degenerada (s)
200.000	128,2	886,21	307	560,7
400.000	171,8	4.176,89	662	3.060,5

Tabela – tempo de montagem: listas e árvores

Tamanho	Lista não ordenada (ns)	Lista ordenada (ns)	Árvore balanceada (ns)	Árvore degenerada (ns)
50.000	5.432.309	9.141.336	219.564	3.416.913
100.000	7.446.970	10.828.974	227.037	10.566.543
200.000	8.097.667	17.345.698	264.276	15.970.010
400.000	10.779.846	34.486.922	213.352	24.577.554

Tabela – pesquisa de pior caso pelo telefone: listas e árvores

Tamanho	Lista não ordenada (ns)	Lista ordenada (ns)	Árvore balanceada (ns)	Árvore degenerada (ns)
50.000	4.269.648	9.174.849	7.875	3.244.465
100.000	6.034.364	9.702.087	7.825	9.252.237
200.000	7.338.313	16.827.141	8.326	15.589.482
400.000	10.043.767	35.195.429	8.526	24.554.280

Tabela – remoção de pior caso pelo telefone: listas e árvores

Montagem. A árvore degenerada faz $n \cdot (n - 1)/2$ comparações, $O(n^2)$, a mesma conta da lista ordenada do Trabalho 1. Por isso as duas levam segundos, e o tempo multiplica por quatro ou mais a cada vez que o arquivo dobra (ver 3.2.3). A árvore balanceada é $O(n \log n)$ e leva milissegundos, como a lista não ordenada.

A árvore degenerada foi mais rápida que a lista ordenada em todos os tamanhos (3.060,5 s contra 4.176,89 s com 400.000), mas ficou na mesma ordem de grandeza. Faz sentido, porque as duas fazem cerca de $n^2/2$ comparações. No Trabalho 1 eram duas listas ordenadas, e cada inserção percorria metade de cada uma; aqui só a árvore de telefones é degenerada. Já a árvore balanceada foi de 2,2 a 3,9 vezes mais lenta que a lista não ordenada (662 ms contra 171,8 ms com 400.000). A árvore faz cerca de $\log_2 n$ comparações em cada inserção, e a lista não ordenada não compara nada, só coloca o contato no fim.

Pesquisa e remoção. Nas listas e na árvore degenerada, as duas operações são $O(n)$. Na árvore balanceada são $O(\log n)$: com 400.000 contatos, são no máximo 19 comparações, contra 400.000. Nas medições com 400.000 contatos, a remoção levou 8.526 ns na árvore balanceada, 10.043.767 ns na lista não ordenada e 24.554.280 ns na árvore degenerada. A árvore balanceada foi cerca de 1.178 vezes mais rápida, e a degenerada ficou na mesma ordem de grandeza das listas. Um cuidado: nos dados do Trabalho 1, a pesquisa nas listas cresceu bem menos que o tamanho do arquivo (na lista não ordenada, foi de 5,4 ms para 10,8 ms com 8 vezes mais contatos). Provavelmente é o aquecimento da JVM, porque lá cada

pesquisa foi medida uma vez só. Na árvore degenerada, medida do mesmo jeito, a pesquisa foi de 3,4 ms para 24,6 ms.

Conclusão da comparação: A árvore só é melhor que a lista quando está balanceada. Com o mesmo código, a árvore montada com o arquivo ordenado teve desempenho de lista na montagem, na pesquisa e na remoção. Como não dá para controlar a ordem em que os dados chegam, precisamos de uma árvore que se rebalanceie sozinha, que é o objetivo da árvore AVL na próxima parte do trabalho.